

我会按“模块拆分 + 版本演进 + 工程依赖关系”的方式来组织，这样后面你可以直接把它变成项目 roadmap 或 Cursor agent 的开发任务。

已思考若干秒 ›

可以。现在我们把它从“产品想法”转成“工程项目”。

我建议这个项目不要按页面拆，而要按**能力模块**拆。因为你的核心不是普通 App，而是一个长期演进的 **时间管理 Agent 系统**。

一、项目整体模块拆分

我建议拆成 10 个核心模块：

1. 用户与本地配置模块
2. 任务 Todo 模块
3. 时间表 Timetable 模块
4. 排程引擎 Schedule Engine
5. Agent 工具调用模块
6. 对话与意图理解模块
7. Heartbeat 主动触发模块
8. 通知与提醒模块
9. 记忆与习惯学习模块
10. 多端同步与云后端模块

其中前 8 个是桌面端 MVP 的核心，后 2 个是中后期能力。

二、模块解释

1. 用户与本地配置模块

这个模块负责保存用户的基础偏好。

包括：

- 用户昵称
- 每日可用时间
- 睡眠时间
- 工作日/周末规则
- 默认任务时长
- 默认休息间隔
- 提醒偏好

是否允许自动重排
是否需要操作确认

这个模块看起来简单，但很重要，因为排程引擎和 Agent 都要依赖它。

例如用户设置：

我通常 00:30 睡觉
我上午适合学习
我不希望晚上 10 点后安排重任务
每 90 分钟任务后需要休息 10 分钟

这些都会影响自动排程。

2. 任务 Todo 模块

这是最基础的数据模块。

任务至少需要这些字段：

```
id
title
description
deadline
estimated_duration
actual_duration
priority
status
category
is_flexible
can_split
created_at
updated_at
```

任务状态建议设计成：

inbox	未安排
scheduled	已安排
in_progress	进行中
completed	已完成
missed	错过
cancelled	取消

这里要注意，**Todo** 和 **Timetable** 不能混成一个东西。

Todo 是“我要做什么”。

Timetable 是“我什么时候做”。

一个任务可以对应一个或多个时间块。

例如：

任务：写数据挖掘报告，预计 4 小时

时间块 1：周三 14:00-16:00

时间块 2：周四 10:00-12:00

所以 Todo 模块和 Timetable 模块必须分开。

3. 时间表 Timetable 模块

这个模块负责管理具体时间块。

时间块字段建议包括：

```
id
task_id
title
start_time
end_time
type
status
is_locked
source
created_at
updated_at
```

其中 type 可以是：

task	普通任务块
event	固定事件，比如课程、会议
break	休息
routine	固定习惯
buffer	缓冲时间

source 可以是：

manual	用户手动创建
agent	Agent 创建
schedule	排程引擎创建
calendar	外部日历同步

is_locked 很关键。

如果用户手动锁定某个时间块，Agent 不应该随便改。

例如：

```
20:00-21:00 健身, locked = true
```

那么重排时不能把它挪走。

4. 排程引擎 Schedule Engine

这是整个项目最核心的非 AI 模块。

它负责把 todo list 安排进 timetable。

第一版不需要复杂 AI，先用规则算法。

输入：

```
待安排任务
已有时间块
用户偏好
日期范围
当前时间
```

输出：

```
新的时间块安排
冲突列表
未能安排的任务
重排建议
```

第一版规则可以这样：

1. 固定事件和 locked block 不可移动
2. deadline 越近，任务优先级越高
3. priority 越高，越早安排
4. 长任务优先找连续空闲时间
5. 可拆分任务可以切成多个 block
6. 每个任务之间插入休息或 buffer
7. 安排不下的任务进入 conflict list

排程引擎必须是**确定性的**，不要完全交给 LLM。

LLM 可以帮用户理解需求，但真正排时间应该由 Schedule Engine 执行。

5. Agent 工具调用模块

这是 Agent 和系统之间的桥。

Agent 不能直接改数据库，而是只能调用工具。

例如：

```
create_task()
update_task()
delete_task()
list_tasks()
create_time_block()
move_time_block()
delete_time_block()
reschedule_day()
mark_task_completed()
get_today_plan()
explain_schedule()
```

这个模块的价值是保证 Agent 行为可控。

用户说：

“帮我把今晚的英语复习推迟到明天。”

Agent 不应该自己编 SQL，而是调用：

```
list_tasks(keyword="英语复习")
update_task(task_id=xxx, date=明天)
reschedule_day(date=明天)
```

后期要加权限控制：

```
低风险操作：直接执行
中风险操作：需要确认
高风险操作：二次确认
```

6. 对话与意图理解模块

这个模块负责用户和 Agent 的自然语言交互。

用户可能说：

```
明天下午帮我安排两小时写 HBase 实验报告
今天晚上还有什么？
我不想做这个了，换到周末
```

这个任务已经完成了
今天状态不好，帮我重新安排轻松一点

系统要把这些话转换成结构化 intent。

例如：

```
{
  "intent": "create_and_schedule_task",
  "task": {
    "title": "写 HBase 实验报告",
    "duration_minutes": 120,
    "preferred_time": "tomorrow afternoon"
  }
}
```

第一版可以先支持少量 intent：

创建任务
查询任务
修改任务
删除任务
标记完成
安排任务
重排今天
解释安排

不要一开始支持太复杂的自由对话。

7. Heartbeat 主动触发模块

这是你产品区别于 SkedPal 的关键模块。

它不是等待用户说话，而是周期性检查系统状态。

例如每 15 分钟检查一次：

有没有任务快开始？
有没有任务应该结束？
有没有 todo 没有安排？
有没有 deadline 快到了？
当前计划是否已经不现实？
用户是否长时间没有反馈？

Heartbeat 的输出不是一定要通知用户，而是生成一个 decision：

```
do_nothing
silent_update
show_badge
send_notification
open_chat_prompt
ask_confirmation
```

例如：

检测到：一个任务已经过了预计完成时间，但用户没有确认完成。
动作：弹出轻量询问。

Agent 说：

“刚才的数据库作业完成了吗？完成 / 还差一点 / 没开始 / 重新安排。”

这个模块就是产品的灵魂之一。

8. 通知与提醒模块

这个模块负责把 Agent 的判断传达给用户。

提醒类型：

```
任务开始提醒
任务结束确认
deadline 风险提醒
未安排任务提醒
计划冲突提醒
重排建议提醒
每日开始提醒
每日结束复盘
```

通知不要只是弹窗，而要带动作按钮。

例如：

现在该开始写报告了

[开始专注] [推迟 30 分钟] [重新安排] [跳过]

这样提醒本身就是 Agent 工作流的一部分。

9. 记忆与习惯学习模块

这个模块负责长期智能化。

第一版可以先不做复杂模型，只做行为日志。

记录：

- 任务计划开始时间
- 实际开始时间
- 计划结束时间
- 实际结束时间
- 是否完成
- 是否延期
- 延期次数
- 任务类型
- 用户手动调整行为
- 用户接受/拒绝 Agent 建议

之后可以统计出：

- 用户经常低估写作任务时间
- 用户上午完成率更高
- 用户晚上容易拖延
- 用户不喜欢连续 2 小时以上的任务
- 用户经常把编程任务延后

这个模块后期会影响排程引擎。

10. 多端同步与云后端模块

这个模块不要一开始做。

等桌面端数据模型稳定后，再把本地 SQLite 抽象成云端数据库。

未来模块包括：

- 用户登录
- 云端 PostgreSQL
- 设备同步
- 冲突解决
- WebSocket 实时更新
- 移动端 API
- 外部日历接入

核心难点是同步冲突。

例如桌面端和手机端同时修改同一个任务，就需要事件模型来处理。

三、版本规划与模块对应关系

下面把模块和版本结合起来。

V0：本地基础版

目标

先做一个可以本地运行的“Structured-like 时间管理工具”。

不做 AI，不做云，不做复杂 Agent。

核心目标：

- Todo 可以创建
- 任务可以安排进时间表
- 时间轴可以展示
- 数据可以本地保存

需要完成的模块

模块	V0 是否需要
用户与本地配置模块	简化版
Todo 模块	必须
Timetable 模块	必须
Schedule Engine	简化版
Agent 工具调用模块	不需要
对话模块	不需要
Heartbeat 模块	不需要
通知模块	简化版
记忆模块	不需要
云同步模块	不需要

V0 功能清单

创建任务
编辑任务
删除任务
设置 <code>deadline</code>
设置预计时长
查看今日时间轴
手动拖拽任务块
本地 <code>SQLite</code> 保存
简单提醒

V0 技术建议

Tauri / Electron
React + TypeScript
<code>SQLite</code>
本地状态管理 Zustand / Redux

V0 验收标准

用户可以创建一个任务
用户可以把任务安排到某个时间段
用户可以在时间轴看到任务
用户关闭应用再打开，数据仍然存在
用户可以修改或删除任务

V1：Agent 工具调用版

目标

让用户可以通过自然语言控制任务和时间表。

这版开始出现真正的 Agent 能力，但还不需要很强的主动性。

需要完成的模块

模块	V1 是否需要
用户与本地配置模块	基础版
Todo 模块	完整基础版
Timetable 模块	完整基础版

模块

Schedule Engine

Agent 工具调用模块

对话模块

Heartbeat 模块

通知模块

记忆模块

云同步模块

V1 是否需要

基础自动排程

必须

必须

不需要

基础版

行为日志雏形

不需要

V1 功能清单

自然语言创建任务

自然语言查询今日计划

自然语言修改任务

自然语言删除任务

自然语言标记完成

Agent 调用工具修改数据库

一键自动安排今日任务

一键重排今天剩余时间

Agent 解释为什么这样安排

示例交互

用户：

明天下午帮我安排两个小时写数据挖掘报告。

Agent：

我已创建任务“写数据挖掘报告”，预计 2 小时。

明天下午 14:00-16:00 有空，我已经帮你安排进去。

V1 验收标准

用户说自然语言后，系统能正确创建任务

Agent 不能绕过工具直接改数据

每次 Agent 操作都有日志